

FastAPI for AI/ML Systems

1. Overview

FastAPI is a modern Python framework for building HTTP APIs. It is based on standard Python type hints and uses:

- **Starlette** for HTTP handling, routing, middleware, WebSockets, and asynchronous execution.
- **Pydantic** for parsing, validating, and serializing data.
- **OpenAPI** and **JSON Schema** for machine-readable API documentation.

In an AI/ML system, FastAPI usually sits between a trained model and its consumers:

```
Web/mobile client -> HTTP request -> FastAPI -> preprocessing -> model inference
                  <- HTTP response <- serialization <- prediction/postprocessing
```

FastAPI does not train a model. It exposes model capabilities through endpoints such as:

- `POST /predict` for online inference.
- `POST /batch-predict` for small synchronous batches.
- `GET /health/live` and `GET /health/ready` for deployment probes.
- `GET /model-info` for model metadata.
- WebSocket or streaming endpoints for token-by-token LLM output.

It is useful because type annotations define both runtime validation and API documentation. A request model such as `Applicant(features: list[float])` becomes input validation, a JSON Schema, and interactive Swagger documentation with little duplicated code.

Typical real-world uses include online fraud scoring, recommendation APIs, computer-vision upload services, NLP classification, embedding generation, RAG gateways, LLM streaming, internal model platforms, and model-serving microservices.

FastAPI is a strong choice when the surrounding stack is Python and inference logic uses libraries such as scikit-learn, PyTorch, TensorFlow, Transformers, or ONNX Runtime. For very high-throughput GPU serving, specialized servers such as NVIDIA Triton, vLLM, or TensorFlow Serving may run the model while FastAPI acts as an orchestration or business-logic layer.

2. Intuition

Think of a trained model as a specialist who understands vectors, tensors, and probability scores but does not understand web requests. FastAPI acts as the receptionist:

1. It accepts a request at a known address such as `/predict`.
2. It checks that the request contains the required fields in the correct format.
3. It transforms the request into the representation expected by the model.
4. It calls the model.

5. It converts the result into a stable JSON response.
6. It returns an appropriate HTTP status code.

For example, a placement prediction model may expect a numeric matrix with columns in this exact order:

```
[cgpa, internships, projects, aptitude_score]
```

A client sends:

```
{
  "cgpa": 8.4,
  "internships": 2,
  "projects": 4,
  "aptitude_score": 81
}
```

FastAPI validates the ranges, the preprocessing pipeline converts the values to a model-ready row, and the model returns a probability. The API might respond:

```
{
  "label": "placed",
  "probability": 0.8731,
  "model_version": "placement-lr-v1"
}
```

The important idea is that an API is a **contract**, not merely a Python function exposed over the network. Clients depend on field names, types, status codes, error shapes, latency, and versioning behavior.

3. Prerequisites

Python knowledge

- Functions, classes, exceptions, imports, and context managers.
- Type hints such as `str`, `list[float]`, `dict[str, int]`, and `X | None`.
- `async def`, `await`, and the difference between blocking and non-blocking work.
- Virtual environments and dependency installation.

Web and API fundamentals

- HTTP methods: `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`.
- URLs, paths, query parameters, headers, cookies, and request bodies.
- JSON serialization.
- HTTP status codes such as `200`, `201`, `400`, `401`, `404`, `409`, `422`, and `500`.
- REST conventions and idempotency.

ML engineering knowledge

- Training versus inference.
- Preprocessing pipelines and feature order.
- Model serialization with `joblib`, `pickle`, TorchScript, ONNX, or framework-specific formats.
- Classification probabilities, thresholds, regression outputs, and basic evaluation metrics.
- Training-serving skew and schema compatibility.

Production basics

- Environment variables and secret management.
- Structured logging, health checks, metrics, containers, and process workers.
- Authentication and authorization.
- Unit and integration testing.

No advanced mathematics is required to learn FastAPI. The most relevant quantitative ideas are latency, throughput, queueing, classification thresholds, and resource utilization.

4. Core Concepts

4.1 ASGI and the application server

What it means: FastAPI is an **ASGI** application. ASGI, the Asynchronous Server Gateway Interface, defines how an asynchronous Python web application communicates with a server such as Uvicorn or Hypercorn.

Why it matters: ASGI supports long-lived connections, WebSockets, and efficient handling of many requests that spend time waiting for network or database I/O.

Simple example: The command below starts one Uvicorn process and imports the object named `app` from `app.py`:

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Common interview angle: WSGI is primarily synchronous; ASGI supports asynchronous request handling and protocols such as WebSockets. ASGI does not automatically make CPU-heavy model inference faster.

4.2 Path operations and HTTP methods

What it means: A path operation associates a Python function with an HTTP method and path.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

Why it matters: A clean route design communicates resource semantics and makes APIs predictable.

Simple example: `GET /models/42` reads model metadata, while `POST /predictions` creates a prediction result.

Common interview angle: `GET` should be safe and normally idempotent. `POST` is commonly used for inference because request bodies may be large and predictions may be logged or billed, even when the mathematical operation itself is read-only.

4.3 Path parameters, query parameters, headers, and bodies

What it means: FastAPI decides where a value comes from using the route template, its type, and explicit helpers.

```
from typing import Annotated
from fastapi import Header, Query

@app.get("/models/{model_id}")
def model_info(
    model_id: str,
    include_metrics: bool = Query(False),
    x_request_id: Annotated[str | None, Header()] = None,
):
    return {
        "model_id": model_id,
        "include_metrics": include_metrics,
        "request_id": x_request_id,
    }
```

Why it matters: Correct placement produces a clear API contract and avoids ambiguous input handling.

Simple example: Resource identity belongs in the path, optional filtering belongs in the query string, metadata belongs in headers, and structured inference data belongs in the body.

Common interview angle: A Pydantic model parameter is interpreted as a request body; a scalar not present in the path is usually treated as a query parameter.

4.4 Pydantic request validation

What it means: A Pydantic `BaseModel` declares a data schema and validates incoming data before the endpoint runs.

```
from pydantic import BaseModel, Field

class PredictionRequest(BaseModel):
    cgpa: float = Field(ge=0, le=10)
    projects: int = Field(ge=0, le=100)
```

Why it matters: Models should never receive malformed shapes, impossible ranges, or missing fields. Validation at the API boundary prevents unclear downstream errors.

Simple example: A `cgpa` of `12` fails validation and FastAPI returns a structured `422 Unprocessable Entity` response.

Common interview angle: Explain the difference between syntactically valid JSON and semantically valid input. Pydantic validates the latter according to the declared schema.

4.5 Response models and serialization

What it means: `response_model` or a return type declares the expected output schema.

```
class PredictionResponse(BaseModel):
    label: str
    probability: float

@app.post("/predict", response_model=PredictionResponse)
def predict(payload: PredictionRequest):
    return {"label": "placed", "probability": 0.87, "internal_debug": "hidden"}
```

Why it matters: Response validation catches server-side contract violations and output filtering prevents accidental leakage of internal fields.

Simple example: `internal_debug` is omitted because it is not part of `PredictionResponse`.

Common interview angle: Request validation protects the server from bad client input; response validation protects clients from server implementation mistakes.

4.6 Dependency injection

What it means: `Depends` lets endpoint functions declare reusable dependencies such as authentication, database sessions, configuration, or service objects.

```
from typing import Annotated
from fastapi import Depends, Header, HTTPException

def require_api_key(x_api_key: Annotated[str | None, Header()] = None) -> str:
```

```

if x_api_key != "development-key":
    raise HTTPException(status_code=401, detail="Invalid API key")
return x_api_key

@app.post("/secure-predict")
def secure_predict(
    payload: PredictionRequest,
    _: Annotated[str, Depends(require_api_key)],
):
    return {"accepted": True}

```

Why it matters: Dependencies centralize cross-cutting logic and can be overridden in tests.

Simple example: One authentication dependency can protect many routes.

Common interview angle: FastAPI constructs a dependency graph, resolves sub-dependencies, caches dependency results within a request by default, and passes results into the endpoint.

4.7 `async def` versus `def`

What it means: Use `async def` when the function awaits asynchronous I/O. Use ordinary `def` for blocking libraries; FastAPI runs synchronous route functions in a thread pool.

```

@app.get("/async-example")
async def async_example():
    result = await async_http_client.get("https://service/internal")
    return result.json()

```

Why it matters: Calling blocking I/O directly inside `async def` blocks the event loop and delays unrelated requests.

Simple example: `await asyncio.sleep(1)` yields control; `time.sleep(1)` blocks the executing thread.

Common interview angle: CPU-bound inference does not become non-blocking merely because the route uses `async def`. Use process-level parallelism, a job queue, a dedicated inference server, or framework-specific batching.

4.8 Application lifespan and model loading

What it means: A lifespan context manager performs startup and shutdown work.

```

from contextlib import asynccontextmanager
from fastapi import FastAPI

resources: dict[str, object] = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    resources["model"] = load_model()

```

```
yield
resources.clear()

app = FastAPI(lifespan=lifespan)
```

Why it matters: Loading a large model per request is slow and wasteful. Loading once per process amortizes startup cost.

Simple example: A 2 GB model loaded by four worker processes may consume roughly 8 GB plus runtime overhead unless memory sharing or an external model server is used.

Common interview angle: "Load once" means once per process, not once per cluster or necessarily once per machine.

4.9 Errors and status codes

What it means: Expected client or domain errors should use `HTTPException`; unexpected errors should be logged and handled centrally.

```
from fastapi import HTTPException

if model_version not in available_versions:
    raise HTTPException(status_code=404, detail="Model version not found")
```

Why it matters: Status codes let clients distinguish retryable failures, invalid requests, authentication failures, and server faults.

Simple example: Return `413 Payload Too Large` for an oversized image rather than allowing memory exhaustion.

Common interview angle: Do not convert every exception into `200 OK` with an `{ "error": ... }` body. HTTP status codes are part of the API contract.

4.10 Middleware

What it means: Middleware wraps every request and response. It is useful for request IDs, timing, CORS, compression, and common headers.

```
import time
import uuid
from fastapi import Request

@app.middleware("http")
async def add_request_metadata(request: Request, call_next):
    request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
```

```
response.headers["X-Process-Time"] = f"{time.perf_counter() - start:.6f}"
return response
```

Why it matters: Cross-cutting behavior stays consistent across routes.

Common interview angle: Middleware operates at the HTTP request level, while dependencies can access parsed parameters and are better for route-specific concerns such as authorization.

4.11 Routers and application organization

What it means: `APIRouter` groups related routes and can apply prefixes, tags, or shared dependencies.

```
from fastapi import APIRouter

router = APIRouter(prefix="/v1", tags=["predictions"])

@router.post("/predict")
def predict_v1(payload: PredictionRequest):
    ...

app.include_router(router)
```

Why it matters: Routers keep a growing service navigable without creating a new application for every feature.

Common interview angle: Use routers for modularity; use separate services only when deployment, ownership, scaling, or reliability boundaries genuinely differ.

4.12 OpenAPI and automatic documentation

What it means: FastAPI generates an OpenAPI schema, Swagger UI at `/docs`, and ReDoc at `/redoc` by default.

Why it matters: Clients can inspect and generate code from the API contract. Documentation remains close to executable types.

Simple example: Field descriptions, constraints, examples, route summaries, and response models appear automatically in the docs.

Common interview angle: Automatic docs are only accurate when schemas and status responses are declared correctly. Generated documentation does not replace careful API design.

4.13 File uploads, streaming, and WebSockets

What it means: `UploadFile` streams file content through a spooled temporary file, `StreamingResponse` streams output chunks, and WebSockets support bidirectional long-lived communication.

Why it matters: These mechanisms fit image inference, audio transcription, large downloads, and token streaming.

Simple example: Prefer `UploadFile` over reading an entire large upload into a `bytes` field.

Common interview angle: WebSockets add operational complexity. Server-Sent Events or an HTTP streaming response may be enough for one-way LLM token streaming.

4.14 Background tasks

What it means: `BackgroundTasks` runs small work after the response is sent within the same application process.

Why it matters: It is useful for lightweight actions such as writing an audit record or sending a non-critical notification.

Simple example: Queue logging after returning a prediction.

Common interview angle: It is not a durable distributed job queue. Long-running training, batch inference, or critical work belongs in Celery, Dramatiq, RQ, a cloud queue, or an orchestration platform.

4.15 Testing

What it means: `TestClient` sends requests to the ASGI application without starting an external server.

```
from fastapi.testclient import TestClient

client = TestClient(app)

def test_health():
    response = client.get("/health/live")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

Why it matters: API tests verify routing, validation, serialization, dependencies, and error behavior together.

Common interview angle: Override external dependencies and model services in tests rather than loading a production model or calling a real database.

4.16 Security

What it means: Authentication establishes identity; authorization decides what that identity may do. FastAPI provides helpers for API keys, OAuth2, bearer tokens, and security schemes.

Why it matters: Prediction endpoints may expose sensitive models, personal data, costly GPUs, or paid third-party APIs.

Simple example: Validate a bearer token, check scopes, cap payload size, rate-limit callers at a gateway, and use HTTPS.

Common interview angle: CORS is a browser access policy, not authentication. A permissive CORS setting does not secure an endpoint.

5. Algorithm / Working Process

FastAPI is a web framework rather than a learning algorithm, so its “working process” is the lifecycle of a request.

5.1 Application startup

1. Uvicorn starts an event loop and imports the FastAPI application.
2. FastAPI registers routes, middleware, exception handlers, and dependency graphs.
3. The lifespan startup code loads configuration, model artifacts, tokenizer, preprocessing pipeline, and other shared resources.
4. Readiness becomes successful only after required resources are usable.

5.2 Inference request lifecycle

1. **Receive:** The ASGI server accepts an HTTP connection and creates an ASGI request scope.
2. **Route:** FastAPI matches the HTTP method and URL path.
3. **Extract:** Path parameters, query values, headers, cookies, forms, files, and JSON bodies are extracted.
4. **Validate:** Pydantic parses types and applies field/model constraints.
5. **Resolve dependencies:** Authentication, database sessions, service objects, and nested dependencies are evaluated.
6. **Preprocess:** Application code converts validated domain fields into the exact feature representation used in training.
7. **Infer:** The loaded model computes a prediction.
8. **Postprocess:** Scores become labels, ranked results, boxes, tokens, or business decisions.
9. **Serialize:** FastAPI validates and serializes the declared response.
10. **Return:** Middleware adds headers, emits timing/log data, and the ASGI server sends the response.

5.3 Input, processing, and output for an ML endpoint

Stage	Placement prediction example
Input	JSON with CGPA, internships, projects, and aptitude score
Validation	Type checks and domain ranges
Preprocessing	Stable feature order, scaling/encoding inside a saved pipeline
Model processing	Logistic regression computes a probability
Postprocessing	Apply a configurable decision threshold
Output	Label, probability, threshold, and model version

5.4 Training process

Training normally happens outside the FastAPI process:

1. Version and validate a dataset.
2. Split data without leakage.
3. Fit preprocessing and the model as one pipeline.

4. Evaluate and select an operating threshold.
5. Serialize the complete inference artifact.
6. Record model version, feature schema, metrics, and library versions.
7. Deploy the immutable artifact with or behind the API.

Training inside a request handler is usually incorrect because it makes latency unpredictable, consumes shared resources, and creates concurrent state-management problems.

5.5 Inference process

At inference time, do not refit preprocessing. Load the fitted pipeline, construct one or more rows in the training schema, call `predict_proba` or the appropriate model method, and return a stable response. Log operational metadata, but avoid logging raw personal or secret data.

6. Mathematical Foundation

FastAPI itself has no loss function. The relevant mathematics comes from the served model and from system performance.

6.1 Example model: logistic regression

For a feature vector $x \in \mathbb{R}^d$, logistic regression computes:

$$z = w^T x + b$$

and converts the score to a probability using the sigmoid function:

$$p(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The API converts the probability into a class using a threshold τ :

$$\hat{y} = \begin{cases} 1, & p \geq \tau \\ 0, & p < \tau \end{cases}$$

The default $\tau = 0.5$ is not universally optimal. A hiring-screening or fraud system should choose a threshold based on validation data, asymmetric error costs, fairness analysis, and business policy.

The usual training loss is binary cross-entropy:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

This loss belongs to offline training. The FastAPI service only evaluates the learned function unless online learning is explicitly designed.

6.2 Latency

End-to-end latency can be approximated as:

$$L_{total} = L_{network} + L_{queue} + L_{validation} + L_{preprocess} + L_{inference} + L_{postprocess} + L_{serialization}$$

Average latency alone hides tail behavior. Production services commonly track percentiles:

- **p50**: half of requests are faster.
- **p95**: 95% of requests are faster.
- **p99**: captures severe tail latency important to user experience and timeouts.

6.3 Throughput

Throughput is completed work per unit time:

$$T = \frac{N_{completed}}{\Delta t}$$

It is usually reported as requests per second (RPS), samples per second, or tokens per second. For batching, distinguish request throughput from sample throughput.

6.4 Little's Law

For a stable system:

$$N = \lambda W$$

where:

- N is the average number of requests in the system.
- λ is arrival rate or throughput.
- W is average time in the system.

If a service handles 100 requests/s with an average latency of 0.2 s, it has roughly:

$$N = 100 \times 0.2 = 20$$

requests concurrently in flight. This is useful for sizing connection pools, concurrency limits, and load tests.

6.5 Utilization and queueing

A simplified utilization estimate is:

$$\rho = \frac{\lambda}{c\mu}$$

where λ is arrival rate, c is the number of parallel servers/workers, and μ is service rate per worker. As ρ approaches 1, queueing delay often rises sharply. Therefore, running consistently at 100% CPU or GPU utilization may produce unacceptable p99 latency.

6.6 Batching trade-off

For batch size B , per-sample compute time is:

$$C_{sample}(B) = \frac{C_{batch}(B)}{B}$$

Batching can improve GPU utilization and sample throughput, but adds waiting time while a batch is formed:

$$L_{request} \approx L_{batch_wait} + C_{batch}(B) + L_{overhead}$$

Large batches may increase throughput while violating latency objectives. Dynamic batching uses a maximum batch size and a short maximum wait time.

6.7 Availability and error rate

Observed success rate is:

$$S = \frac{N_{successful}}{N_{total}}$$

Operational error rate is:

$$E = 1 - S = \frac{N_{failed}}{N_{total}}$$

Separate client errors (4xx) from server errors (5xx). A rise in 422 responses may indicate a client/schema integration problem, while a rise in 500 responses indicates a server-side failure.

6.8 Model quality versus service quality

An accurate model can still create a poor product if the service is slow or unreliable. Monitor both:

Model metrics	Service metrics
Precision, recall, F1, ROC-AUC, MAE	RPS, p50/p95/p99 latency, 4xx/5xx, saturation
Calibration and threshold behavior	CPU/GPU/memory, queue depth, timeouts
Drift and slice performance	Availability, readiness, dependency failures

7. Practical Implementation

The following example trains a tiny placement classifier only when no artifact exists, then serves it through a validated FastAPI endpoint. In a real project, training should be a separate pipeline; the fallback keeps this learning example runnable end to end.

7.1 Install dependencies

```
python -m pip install fastapi "uvicorn[standard]" scikit-learn joblib numpy httpx
pytest
```

7.2 app.py

```
from __future__ import annotations

import logging
import os
import time
import uuid
from contextlib import asynccontextmanager
```

```

from pathlib import Path
from typing import Annotated

import joblib
import numpy as np
from fastapi import Depends, FastAPI, Header, HTTPException, Request, status
from pydantic import BaseModel, ConfigDict, Field
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("placement-api")

MODEL_PATH = Path(os.getenv("MODEL_PATH", "placement_model.joblib"))
MODEL_VERSION = os.getenv("MODEL_VERSION", "placement-lr-v1")
API_KEY = os.getenv("API_KEY", "development-key")
DECISION_THRESHOLD = float(os.getenv("DECISION_THRESHOLD", "0.5"))

class PlacementRequest(BaseModel):
    """Public request contract."""

    model_config = ConfigDict(extra="forbid")

    cgpa: float = Field(ge=0, le=10, examples=[8.4])
    internships: int = Field(ge=0, le=20, examples=[2])
    projects: int = Field(ge=0, le=100, examples=[4])
    aptitude_score: float = Field(ge=0, le=100, examples=[81])

class PlacementResponse(BaseModel):
    label: str
    probability: float = Field(ge=0, le=1)
    threshold: float = Field(ge=0, le=1)
    model_version: str

class HealthResponse(BaseModel):
    status: str

class Predictor:
    """Small adapter that owns feature order and model inference."""

    feature_names = ("cgpa", "internships", "projects", "aptitude_score")

    def __init__(self, pipeline: Pipeline) -> None:
        self.pipeline = pipeline

    def predict(self, request: PlacementRequest) -> PlacementResponse:
        row = np.array(
            [[

```

```

        request.cgpa,
        request.internships,
        request.projects,
        request.apptitude_score,
    ]],
    dtype=np.float64,
)
probability = float(self.pipeline.predict_proba(row)[0, 1])
label = "placed" if probability >= DECISION_THRESHOLD else "not_placed"
return PlacementResponse(
    label=label,
    probability=round(probability, 6),
    threshold=DECISION_THRESHOLD,
    model_version=MODEL_VERSION,
)

```

```
def build_demo_model() -> Pipeline:
```

```
    """Create a deterministic demo artifact; production training lives
    elsewhere."""
```

```

    features = np.array(
        [
            [5.8, 0, 1, 48],
            [6.4, 0, 2, 55],
            [6.9, 1, 2, 61],
            [7.4, 1, 3, 67],
            [7.8, 2, 3, 72],
            [8.1, 1, 4, 75],
            [8.5, 2, 4, 82],
            [8.9, 3, 5, 88],
            [9.2, 3, 6, 92],
            [9.5, 4, 7, 96],
        ],
        dtype=np.float64,
    )
    targets = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1])
    pipeline = Pipeline(
        [
            ("scale", StandardScaler()),
            ("model", LogisticRegression(random_state=42)),
        ]
    )
    return pipeline.fit(features, targets)

```

```
def load_predictor() -> Predictor:
```

```

    if MODEL_PATH.exists():
        pipeline = joblib.load(MODEL_PATH)
        logger.info("Loaded model from %s", MODEL_PATH)
    else:
        pipeline = build_demo_model()
        joblib.dump(pipeline, MODEL_PATH)
        logger.warning("Created demo model at %s", MODEL_PATH)

```

```

        return Predictor(pipeline)

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.predictor = load_predictor()
    app.state.ready = True
    yield
    app.state.ready = False

app = FastAPI(
    title="Placement Prediction API",
    version="1.0.0",
    description="A compact production-style FastAPI inference example.",
    lifespan=lifespan,
)

@app.middleware("http")
async def request_context(request: Request, call_next):
    request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    start = time.perf_counter()
    response = await call_next(request)
    elapsed = time.perf_counter() - start
    response.headers["X-Request-ID"] = request_id
    response.headers["X-Process-Time"] = f"{elapsed:.6f}"
    logger.info(
        "request_id=%s method=%s path=%s status=%s duration=%.6f",
        request_id,
        request.method,
        request.url.path,
        response.status_code,
        elapsed,
    )
    return response

def require_api_key(
    x_api_key: Annotated[str | None, Header()] = None,
) -> None:
    if x_api_key != API_KEY:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or missing API key",
        )

def get_predictor(request: Request) -> Predictor:
    predictor = getattr(request.app.state, "predictor", None)
    if predictor is None:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="Model is not ready",
        )

```

```

    )
    return predictor

@app.get("/health/live", response_model=HealthResponse, tags=["health"])
def liveness() -> HealthResponse:
    return HealthResponse(status="ok")

@app.get("/health/ready", response_model=HealthResponse, tags=["health"])
def readiness(request: Request) -> HealthResponse:
    if not getattr(request.app.state, "ready", False):
        raise HTTPException(status_code=503, detail="Not ready")
    return HealthResponse(status="ready")

@app.post(
    "/v1/predict",
    response_model=PlacementResponse,
    tags=["predictions"],
    dependencies=[Depends(require_api_key)],
)
def predict(
    payload: PlacementRequest,
    predictor: Annotated[Predictor, Depends(get_predictor)],
) -> PlacementResponse:
    return predictor.predict(payload)

```

7.3 Run the API

```
uvicorn app:app --reload
```

Use `--reload` only for local development. Open <http://127.0.0.1:8000/docs> for Swagger UI.

7.4 Call the prediction endpoint

```

curl -X POST "http://127.0.0.1:8000/v1/predict" \
  -H "Content-Type: application/json" \
  -H "X-API-Key: development-key" \
  -d '{"cgpa":8.4,"internships":2,"projects":4,"aptitude_score":81}'

```

Example response:

```

{
  "label": "placed",
  "probability": 0.884721,
  "threshold": 0.5,

```

```
"model_version": "placement-lr-v1"
}
```

7.5 test_app.py

```
from fastapi.testclient import TestClient

from app import app

def test_prediction_contract():
    with TestClient(app) as client:
        response = client.post(
            "/v1/predict",
            headers={"X-API-Key": "development-key"},
            json={
                "cgpa": 8.4,
                "internships": 2,
                "projects": 4,
                "aptitude_score": 81,
            },
        )

        assert response.status_code == 200
        body = response.json()
        assert body["label"] in {"placed", "not_placed"}
        assert 0 <= body["probability"] <= 1
        assert body["model_version"] == "placement-lr-v1"

def test_invalid_cgpa_is_rejected():
    with TestClient(app) as client:
        response = client.post(
            "/v1/predict",
            headers={"X-API-Key": "development-key"},
            json={
                "cgpa": 11,
                "internships": 2,
                "projects": 4,
                "aptitude_score": 81,
            },
        )

        assert response.status_code == 422

def test_missing_api_key_is_rejected():
    with TestClient(app) as client:
        response = client.post(
            "/v1/predict",
            json={
```

```
        "cgpa": 8.4,  
        "internships": 2,  
        "projects": 4,  
        "aptitude_score": 81,  
    },  
)  
  
assert response.status_code == 401
```

Run the tests:

```
pytest -q
```

7.6 Minimal production container

```
FROM python:3.12-slim  
  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY app.py placement_model.joblib ./  
  
USER 10001  
EXPOSE 8000  
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Pin and scan dependencies in a real deployment. Do not bake API keys into the image.

8. Code Explanation

8.1 Configuration

`MODEL_PATH`, `MODEL_VERSION`, `API_KEY`, and `DECISION_THRESHOLD` come from environment variables. This separates deploy-time configuration from source code. The development defaults make the example runnable; production must inject a secret API key and validated configuration.

8.2 Request and response contracts

`PlacementRequest` uses `Field` constraints to reject impossible values before inference.

`ConfigDict(extra="forbid")` rejects unknown fields, catching misspellings such as `aptitute_score` instead of silently ignoring them.

`PlacementResponse` stabilizes the output and ensures probability and threshold remain in `[0, 1]`. The version makes predictions traceable to an artifact.

8.3 Predictor adapter

`Predictor` owns the feature order. This is important because many tabular estimators consume arrays whose columns have no names at inference time. A wrong order can produce plausible but incorrect predictions without throwing an exception.

The full scikit-learn `Pipeline` contains both scaling and classification, preventing the API from implementing a second, potentially inconsistent preprocessing path.

8.4 Demo training and artifact loading

`build_demo_model` exists only to keep the guide executable. Production training should create and register the artifact independently. `load_predictor` loads the artifact once at startup and wraps it in `Predictor`.

Never load untrusted `pickle` or `joblib` artifacts: deserialization can execute code. Accept artifacts only from a trusted build/model registry and verify provenance or checksums.

8.5 Lifespan state

The lifespan function stores the predictor in `app.state`. Readiness is enabled only after loading succeeds. If startup raises an exception, the process should fail rather than serve requests with an absent model.

Each process has independent application state. If four Uvicorn workers start, each usually loads its own predictor.

8.6 Middleware

The middleware creates or propagates a request ID, measures elapsed time with the monotonic high-resolution `perf_counter`, adds diagnostic response headers, and emits one structured-style log line. In a larger service, export latency histograms and counters to an observability system.

8.7 Dependencies

`require_api_key` protects the endpoint. A real system should compare secrets safely through a standard authentication system, rotate credentials, enforce authorization, use TLS, and usually rate-limit at an API gateway.

`get_predictor` isolates resource retrieval and creates a clear `503` response if the model is unavailable. It can be overridden in tests with a fake predictor.

8.8 Health endpoints

- **Liveness** answers: "Is the process alive?" It should be cheap and should not depend on every downstream service, or temporary dependency failures may cause restart loops.
- **Readiness** answers: "Can this instance receive traffic?" It checks whether required resources such as the model are ready.

8.9 Prediction route

The route remains small: FastAPI validates input and dependencies, while `Predictor` performs inference. A synchronous `def` is reasonable for a blocking scikit-learn call. FastAPI runs it in its thread pool, although

Python-level CPU-bound work still faces CPU and GIL constraints.

8.10 Tests

The first test checks the successful contract without depending on one exact probability. The other tests verify trust-boundary behavior: invalid domain input and missing authentication. `TestClient` runs the lifespan context when used with `with`.

9. Training / Evaluation

FastAPI is evaluated as a serving system, while the model behind it is evaluated as an ML system. Both layers must be tested.

9.1 Dataset preparation

For a placement model:

1. Define the prediction time and ensure every feature would be available then.
2. Remove direct leakage such as a final placement result or post-placement salary.
3. Document units, ranges, missing-value meaning, category vocabulary, and feature order.
4. Fit imputers, encoders, and scalers on training data only.
5. Package preprocessing and the estimator together.

9.2 Train/validation/test split

- Use a **stratified split** for imbalanced classification when observations are independent.
- Use a **time-based split** when serving future events from past data.
- Use a **group split** when multiple rows belong to the same student, customer, patient, or device.
- Keep the test set untouched until model and threshold choices are finalized.

9.3 Model metrics

For binary classification, track:

- Precision when false positives are costly.
- Recall when false negatives are costly.
- F1 when a balance is needed.
- PR-AUC for imbalanced positive classes.
- ROC-AUC for ranking ability, with care under severe imbalance.
- Log loss and calibration when probabilities drive decisions.
- Slice metrics across relevant populations to detect uneven performance.

9.4 API and serving metrics

Track:

- Request count and RPS.
- p50, p95, and p99 latency.
- `2xx`, `4xx`, and `5xx` rates by route and model version.
- Queue depth, timeouts, cancellations, and worker saturation.

- CPU, memory, GPU utilization, GPU memory, and accelerator batch size.
- Input drift, output distribution, confidence distribution, and delayed ground-truth quality.

Avoid metric labels with unbounded cardinality, such as raw user IDs or request IDs.

9.5 Overfitting and underfitting

These are properties of the model, not FastAPI:

- **Overfitting:** training performance is strong but validation/test performance is weak.
- **Underfitting:** both training and validation performance are weak.

Serving can introduce a different failure: **training-serving skew**, where online preprocessing, feature definitions, library versions, or category handling differs from training.

9.6 Hyperparameters and operational controls

Model hyperparameters include regularization, depth, learning rate, and decision threshold. Serving controls include:

- Worker/process count.
- Concurrency limit.
- Request timeout.
- Maximum payload size.
- Maximum batch size and batch wait time.
- CPU thread count used by NumPy, BLAS, PyTorch, or ONNX Runtime.
- Model precision such as FP32, FP16, BF16, or INT8.

Tune serving controls with representative load tests rather than rules of thumb alone.

9.7 Improving performance safely

1. Profile and split latency into queue, preprocessing, inference, and serialization.
2. Remove repeated work: load models once and reuse network/database connections.
3. Use vectorized preprocessing and small batches where appropriate.
4. Control nested parallelism; many web workers multiplied by many BLAS threads can oversubscribe CPUs.
5. Use ONNX Runtime, TorchScript/[torch.compile](#), quantization, or a specialized inference server only after measurement.
6. Scale replicas when the service is horizontally scalable.
7. Validate that optimization preserves prediction parity and accuracy.

10. Complexity and Cost

10.1 Framework overhead

For a request with f scalar fields, validation and serialization are roughly $O(f)$. This is usually small compared with transformer or vision inference but may matter for tiny models at very high RPS.

10.2 Model-dependent inference complexity

Model	Approximate inference cost per sample	Memory notes
Linear/logistic regression	$O(d)$	$O(d)$ parameters
Decision tree	$O(h)$	Proportional to nodes
Random forest	$O(Th)$	Proportional to all tree nodes
Dense neural network	Sum of layer matrix-operation costs	Weights plus activations
Transformer	Self-attention is approximately $O(n^2d)$ per layer	KV cache can dominate generation memory
CNN	Depends on spatial size, channels, kernels, and layers	Activations may dominate for batches

Here, d is feature/hidden dimension, h is tree height, T is number of trees, and n is sequence length.

10.3 Training cost

Training cost is external to FastAPI. A serving container should usually contain an immutable model artifact, not a training loop. Separating the two allows independent scaling, reproducibility, approval, and rollback.

10.4 Worker memory

If one process consumes base memory M_b and a model consumes M_m , then w independent workers can require approximately:

$$M_{total} \approx w(M_b + M_m) + M_{shared}$$

Actual sharing depends on operating system, preload strategy, copy-on-write behavior, runtime allocations, and GPU context behavior. GPU models are commonly one process per GPU or managed by a specialized server; blindly multiplying workers can cause out-of-memory failures.

10.5 CPU versus GPU

- Small tabular and linear models often perform best economically on CPU.
- GPU startup and transfer overhead may outweigh benefits for tiny requests.
- Large CNNs and transformers usually benefit from GPUs, especially with batching.
- GPU utilization, VRAM, batch formation, and token throughput matter more than HTTP framework micro-optimizations.

10.6 Cost controls

- Enforce payload and output limits.
- Apply authentication, quotas, and rate limiting.
- Cache only deterministic, non-sensitive, frequently repeated results with a safe key and expiry.
- Autoscale on useful signals such as queue depth or GPU utilization, not CPU alone when serving on accelerators.
- Route requests to appropriately sized models.
- Reject impossible or abusive work before expensive inference.

11. Common Use Cases

1. **Real-time classification:** fraud, churn, moderation, spam, intent, and risk scoring.
 2. **Regression APIs:** demand estimates, ETA, pricing support, and forecasting access.
 3. **Recommendation endpoints:** retrieve candidates, rank them, and return top-k items.
 4. **Computer vision:** upload an image for classification, OCR, segmentation, or detection.
 5. **NLP services:** sentiment, named-entity recognition, translation, summarization, and embeddings.
 6. **RAG systems:** accept a question, retrieve documents, build a prompt, call an LLM, and cite sources.
 7. **LLM gateways:** authentication, quota control, prompt policies, routing, streaming, and audit metadata.
 8. **Audio APIs:** speech-to-text, speaker classification, and audio event detection.
 9. **Feature services:** expose low-latency computed or stored features to online consumers.
 10. **Experiment services:** route traffic between model versions for shadow, canary, or A/B evaluation.
 11. **Model management:** metadata, version information, readiness, and administrative reload operations.
 12. **Internal data applications:** typed service layer between dashboards and ML/business logic.
-

12. Common Mistakes

12.1 Loading the model on every request

This adds disk/network latency, wastes memory bandwidth, and may exhaust resources. Load once during startup per process.

12.2 Training inside the API route

Training is long-running and stateful. Move it to an offline pipeline or durable job system.

12.3 Using `async def` for blocking inference

An `async` declaration does not turn blocking code into asynchronous code. A blocking call inside the event loop stalls other requests. Use `def`, a thread/process offload, or a dedicated model server depending on the workload.

12.4 Assuming more workers always improve throughput

Each worker may duplicate model memory, BLAS threads, and GPU contexts. Excessive workers can increase contention and latency or cause OOM errors.

12.5 Reimplementing preprocessing online

Separate scaler/encoder code easily drifts from training. Serialize a pipeline or share a versioned transformation implementation.

12.6 Wrong feature order

NumPy arrays do not preserve semantic names. Maintain an explicit schema or use a pipeline that accepts named columns and verify feature compatibility.

12.7 Returning NumPy/PyTorch objects directly

JSON cannot directly serialize many framework types. Convert scalars with `.item()` or `float(...)`, and tensors/arrays with `.tolist()` when output size is controlled.

12.8 Ignoring input limits

An unrestricted image, audio file, list, token sequence, or batch can exhaust memory or create denial-of-service risk. Enforce body, file, dimension, batch, and sequence-length limits.

12.9 Hard-coding secrets

Source code and container images are not secret stores. Inject credentials from a managed secret system and rotate them.

12.10 Confusing CORS with security

CORS controls browser behavior. Non-browser clients can still call the API. Use authentication, authorization, TLS, rate limiting, and network policy.

12.11 Logging sensitive inputs

Prompts, resumes, images, tokens, and prediction features may contain personal or confidential data. Log IDs and safe metadata; apply redaction and retention policy.

12.12 Catching every exception and returning 200

This breaks client retry and monitoring behavior. Use meaningful `4xx/5xx` codes and preserve unexpected failures as server errors after safe logging.

12.13 Using `BackgroundTasks` for critical work

The process can terminate before the task completes. Use a durable queue for retrievable or business-critical jobs.

12.14 Performing per-request global mutation

Mutable global state creates race conditions and differs across workers. Keep inference objects read-only or synchronize carefully; put shared durable state in an appropriate external system.

12.15 Returning raw model confidence as certainty

A score of `0.9` is not necessarily a calibrated 90% real-world probability. Validate calibration, monitor drift, and explain threshold semantics.

12.16 Data leakage and bad evaluation

An excellent API cannot rescue a leaked model. Check time, subject, duplicate, and post-outcome leakage; choose splits and metrics that match deployment.

12.17 No versioning or rollback metadata

Without model and API versions, reproducing incidents becomes difficult. Return or log an artifact version and keep deployments rollbackable.

13. Edge Cases / Limitations

13.1 CPU-bound workloads

FastAPI's async strengths mainly help I/O concurrency. Pure Python CPU-heavy preprocessing and inference can block or contend under the GIL. Native libraries may release the GIL, but behavior must be measured.

13.2 Large GPU models

Multiple application workers may duplicate weights and GPU contexts. FastAPI does not provide dynamic batching, tensor parallelism, KV-cache scheduling, or model sharding by itself. Use a specialized inference backend when these dominate the problem.

13.3 Long-running requests

Training, long batch jobs, and hour-long generation are vulnerable to proxy timeouts, disconnects, deployments, and process crashes. Prefer asynchronous job submission with a job ID, durable state, and polling/callbacks.

13.4 Very large uploads and responses

Reading the full body into memory can exhaust the process. Stream uploads, validate content type and size, store objects externally when appropriate, and stream bounded output.

13.5 Client cancellation

A disconnected client does not always stop downstream GPU or external API work automatically. Cancellation must be propagated where the libraries and protocol allow it.

13.6 Partial dependency failure

A model may be healthy while a feature store, vector database, or LLM provider is unavailable. Define timeouts, bounded retries with jitter, fallbacks where valid, circuit breaking at an appropriate layer, and readiness semantics carefully.

13.7 Schema evolution

Adding an optional field is often backward compatible; renaming/removing fields or changing meaning is not. Use explicit API versions or a controlled compatibility strategy.

13.8 Non-thread-safe model objects

Some runtimes, tokenizers, file handles, or custom preprocessors may not be safe for concurrent calls. Confirm library guarantees, isolate state, or use locks/processes while measuring the performance effect.

13.9 Cold starts

Downloading and loading a large model can take minutes. Use immutable images or local caches, startup probes, sufficient deployment grace periods, and capacity that avoids scaling to zero when latency objectives prohibit cold starts.

13.10 Floating-point and reproducibility differences

Hardware, runtime versions, quantization, nondeterministic kernels, and parallel execution can change the last digits or even borderline labels. Test tolerance and decision-boundary behavior rather than assuming bitwise equality.

13.11 FastAPI is not a complete platform

It does not itself provide a model registry, feature store, experiment tracker, autoscaler, secrets manager, distributed queue, API gateway, or full observability backend. Integrate only the infrastructure the service actually needs.

14. Variations

Variation	What changes	When to use	Placement/project importance
Synchronous JSON inference	One request waits for one response	Small/medium models with bounded latency	Essential
Async I/O orchestration	Route awaits databases, vector stores, or model APIs	RAG and services dominated by network I/O	Essential for AI engineering
Batch endpoint	Request contains multiple samples	Clients can aggregate work; vectorized model inference is efficient	Important
Dynamic batching	Server briefly queues requests into accelerator batches	High-throughput GPU serving	Advanced/project/research
Job-based API	POST returns a job ID; client polls or receives callback	Training, large batch inference, long media processing	Important
Streaming HTTP/SSE	Server emits incremental chunks in one direction	LLM tokens and progress events	Important for GenAI
WebSocket API	Bidirectional persistent communication	Interactive voice, live collaboration, two-way streaming	Advanced
File-upload API	Uses multipart and UploadFile	Vision, audio, and document models	Important

Variation	What changes	When to use	Placement/project importance
FastAPI gateway + model server	FastAPI handles auth/business logic; Triton/vLLM serves models	Large models or accelerator scheduling	Strong production design
Serverless deployment	Instances start on demand	Spiky traffic and small artifacts where cold starts are acceptable	Useful but platform-dependent
Microservice per model	Separate deployment and scaling unit	Different ownership, runtimes, reliability, or hardware needs	Use selectively
Multi-model service	One process routes across several loaded models	Small related models with shared lifecycle and enough memory	Advanced; watch resource isolation
REST versioning	Prefixes such as <code>/v1</code> and <code>/v2</code>	Breaking contract changes	Essential API practice
GraphQL wrapper	Schema-driven flexible queries	Clients need flexible data composition, not usually raw inference	Secondary
gRPC backend	Binary schema and streaming between services	Low-latency internal communication and generated clients	Advanced

For placements, be able to build and explain synchronous JSON inference, validation, model lifecycle, testing, authentication, and deployment. For stronger AI engineering roles, also understand async I/O, batching, streaming, observability, GPU-serving separation, and failure handling.

15. Related Topics

FastAPI vs Flask

Flask is a lightweight WSGI framework with a large ecosystem and explicit extension choices. FastAPI is ASGI-native and integrates type-driven validation and OpenAPI generation. Flask can serve ML models successfully; FastAPI usually requires less code for typed API contracts and async endpoints.

FastAPI vs Django REST Framework

Django REST Framework integrates deeply with Django's ORM, authentication, admin, and batteries-included web stack. FastAPI is often lighter for standalone APIs and ML microservices. Choose based on surrounding application needs, not benchmark headlines.

FastAPI vs model servers

Triton, TorchServe, TensorFlow Serving, vLLM, and similar systems focus on optimized model execution, batching, accelerator scheduling, and model lifecycle. FastAPI focuses on the web/API layer. A common architecture uses both.

ASGI vs WSGI

WSGI models a synchronous request-response call. ASGI is asynchronous and supports HTTP plus long-lived protocols. ASGI improves concurrency for waiting-heavy workloads but does not remove CPU/GPU compute limits.

Pydantic and JSON Schema

Pydantic turns Python type declarations into runtime parsing/validation and JSON Schema. FastAPI uses that schema to build OpenAPI documentation and client-facing contracts.

REST and OpenAPI

REST provides design constraints and resource-oriented conventions. OpenAPI describes concrete operations, parameters, request bodies, responses, and security schemes in a machine-readable format.

Uvicorn and Gunicorn

Uvicorn is an ASGI server. Gunicorn is a Unix process manager that can run ASGI workers through appropriate worker integration. Modern container deployments may run one process per container and let the orchestrator manage replicas; process strategy depends on model memory, workload, and platform.

Docker and Kubernetes

Docker packages the application and runtime. Kubernetes or another orchestrator handles scheduling, replica management, probes, rolling updates, service discovery, and resource limits. FastAPI provides health routes; the orchestrator consumes them.

Model registry and MLflow

A model registry tracks artifact versions, stages/aliases, lineage, and metadata. FastAPI loads an approved artifact and should expose or log its version. MLflow is one possible registry and experiment-tracking tool.

Feature stores

Feature stores help keep feature definitions and values consistent between training and online inference. They address a broader problem than request validation: training-serving feature consistency and low-latency feature retrieval.

RAG

A RAG endpoint often performs validation, authentication, query embedding, retrieval, reranking, prompt construction, LLM streaming, and citation formatting. The async portions are usually external I/O; embedding and generation may be delegated to specialized servers.

Observability

Logs describe events, metrics summarize system behavior, and traces follow a request across services. Model monitoring adds drift, data quality, output distribution, calibration, and delayed outcome metrics.

CI/CD and contract testing

CI runs linting, type checks, unit/integration tests, schema compatibility checks, artifact verification, and container scans. CD uses controlled rollout, health signals, canaries, and rollback to reduce deployment risk.

16. Interview Questions

1. What is FastAPI, and why is it popular for ML APIs?

FastAPI is a Python ASGI web framework built around type hints, Starlette, Pydantic, and OpenAPI. It is popular for ML APIs because most ML code is already Python, request/response validation is concise, documentation is generated automatically, and ASGI supports concurrent I/O and streaming patterns.

2. What is the difference between `def` and `async def` in FastAPI?

An `async def` route runs on the event loop and should await non-blocking operations. A normal `def` route is run in a thread pool so blocking code does not directly block the event loop. CPU-heavy inference is not made faster by `async`; it needs appropriate process, accelerator, batching, or model-server architecture.

3. Why should a model be loaded during application startup?

Loading once per process avoids repeated disk/network I/O and initialization on every request. Startup should fail if a required artifact is invalid so the instance never becomes ready with a broken model.

4. How does FastAPI validate request data?

It derives parameter locations and schemas from route declarations and Python types. Pydantic parses the incoming data and applies type, range, pattern, and model-level rules. Invalid bodies usually produce a structured 422 response before endpoint logic executes.

5. What is the difference between a request model and a response model?

A request model constrains client input. A response model constrains and documents server output, can filter undeclared fields, and detects output contract errors. Both contribute JSON Schema to OpenAPI.

6. How would you serve a 10 GB GPU model with FastAPI?

Avoid creating many workers that each load the model. Commonly run a dedicated GPU inference server or one carefully controlled model process, then let FastAPI handle authentication, validation, orchestration, and streaming. Measure batching, VRAM, token throughput, queueing, and tail latency.

7. What is dependency injection in FastAPI?

An endpoint declares required values through `Depends`. FastAPI resolves the dependency graph and supplies results. It is useful for authentication, authorization, configuration, database sessions, and service objects, and makes testing easier through dependency overrides.

8. When would you use `BackgroundTasks`?

Use it for short, non-critical in-process work after returning a response, such as best-effort audit emission. Do not use it for training, long inference, payment-like critical tasks, or work that requires durability and retry; use a real queue and worker system.

9. How do liveness and readiness checks differ?

Liveness indicates the process is functioning and may be restarted if it is not. Readiness indicates the instance can receive traffic. Readiness may depend on successful model loading; liveness should usually avoid fragile downstream checks that could trigger restart loops.

10. Why can too many Uvicorn workers hurt an ML service?

Workers may each load model weights and initialize math-library threads or GPU contexts. This can multiply memory, oversubscribe CPU cores, create GPU contention, and increase p99 latency. Worker count must be load-tested under realistic resource limits.

11. How do you prevent training-serving skew?

Package preprocessing with the model, version the feature schema and artifacts, validate field semantics and units, test known examples end to end, pin compatible dependencies, and monitor online feature/output distributions.

12. How would you test a FastAPI inference endpoint?

Use `TestClient` or an async ASGI client to test success, invalid schemas, authentication, errors, and response contracts. Override model/database dependencies with deterministic fakes for API tests. Add a smaller number of integration tests using the real artifact and golden inputs to verify inference compatibility.

13. What status codes would you use for common failures?

- `400` for malformed domain requests not captured by schema validation.
- `401` for missing/invalid authentication.
- `403` for authenticated but unauthorized callers.
- `404` for missing resources such as a model version.
- `409` for a state conflict.
- `413` for oversized payloads.
- `422` for structurally parseable but invalid request data.
- `429` for rate limits.
- `503` for unavailable capacity or a non-ready dependency.
- `500` for unexpected server faults.

14. How would you improve a slow prediction endpoint?

Profile first. Separate queueing, validation, preprocessing, inference, and serialization. Load artifacts once, vectorize preprocessing, avoid repeated client creation, control thread oversubscription, batch accelerator work, optimize or quantize the model if quality remains acceptable, and scale replicas after identifying the bottleneck.

15. What is the role of middleware?

Middleware wraps the whole HTTP request/response flow and is appropriate for request IDs, timing, common headers, CORS, and tracing. Dependencies are better for route-aware logic such as authorization or obtaining a model service.

16. How would you version a prediction API?

Use a stable contract and introduce a version such as `/v1` when making breaking request or response changes. Version the model separately because many model artifact updates can preserve the same API contract. Log both API and model versions.

17. How should an API return LLM tokens incrementally?

Use an HTTP streaming response or Server-Sent Events for one-way token delivery; use WebSockets only when bidirectional communication is required. Handle disconnects, cancellation, backpressure, timeouts, proxy buffering, and partial-output errors.

18. Is `pickle/joblib` safe for arbitrary model files?

No. Deserializing an untrusted artifact can execute arbitrary code. Load only trusted artifacts from a controlled registry/build pipeline and verify integrity and provenance. Consider safer formats such as ONNX or framework-specific weight formats where appropriate, while recognizing that surrounding code and configuration still require trust.

19. How do you choose a prediction threshold?

Choose it on validation data using the cost of false positives and false negatives, required precision/recall, calibration, capacity constraints, and fairness/slice behavior. Do not assume `0.5` is optimal, and keep the test set untouched until selection is complete.

20. FastAPI benchmarks well. Does that guarantee a fast ML endpoint?

No. Framework overhead may be a tiny part of total latency. Model compute, preprocessing, data transfer, queues, downstream calls, serialization size, and resource contention usually dominate. Benchmark the complete service with representative requests and concurrency.

17. Practice Tasks

17.1 Small coding task

Build a `POST /v1/predict` endpoint for a saved scikit-learn Iris classifier. Require exactly four finite numeric features, return class name and all class probabilities, and reject unknown input fields.

Acceptance checks: valid request returns `200`; missing feature returns `422`; probabilities sum to approximately 1; response includes a model version.

17.2 Dataset-based project

Train a churn classifier on the Telco Customer Churn dataset. Save preprocessing and the estimator as one pipeline, expose it through FastAPI, and document the endpoint with realistic examples.

Evaluation: PR-AUC, recall at a selected precision, calibration, p95 latency, and test coverage for unknown categories and missing values.

17.3 Experiment idea

Compare these deployment configurations under the same load:

1. One worker with one math-library thread.
2. Multiple workers with one thread each.
3. One worker with model-level multithreading.
4. Small request batching.

Measure RPS, p50/p95/p99 latency, CPU utilization, and memory. Explain why the best throughput configuration may not provide the best tail latency.

17.4 Debugging/analysis task

You deploy four workers and memory grows from 2.5 GB to 10 GB. Diagnose whether each process loads its own model. Then propose a solution based on the actual workload: fewer workers, more replicas with adequate memory, CPU copy-on-write where safe, or a dedicated inference server.

17.5 Extension idea

Convert synchronous prediction into an asynchronous job API:

- `POST /jobs` validates input and enqueues durable work.
- `GET /jobs/{id}` returns state and result metadata.
- A worker performs inference.
- Results expire according to a retention policy.
- Duplicate client retries use an idempotency key.

17.6 Security task

Replace the development API key with bearer-token authentication. Add authorization scopes, rate limits at a gateway, a maximum request size, and tests for 401, 403, 413, and 429 behavior.

17.7 Schema-evolution task

Create `/v2/predict` with a renamed field and additional optional context while keeping `/v1` compatible. Generate both OpenAPI schemas and write contract tests showing which changes are backward compatible.

18. Project Ideas

Project 1: Production-Style Placement Prediction Service

What it does: Predicts placement likelihood, returns calibrated probability and decision metadata, supports model versioning, and exposes health and metrics endpoints.

- Tech stack:** FastAPI, Pydantic, scikit-learn/XGBoost, Pandas, joblib or MLflow, Docker, pytest, Prometheus-compatible metrics.
- Dataset suggestion:** A public campus placement dataset, or a carefully documented synthetic dataset if real features are too limited. Explicitly discuss bias, sensitive attributes, proxy variables, and why the tool must support rather than automate high-stakes hiring decisions.
- Resume value:** Demonstrates end-to-end ML engineering: leakage-aware training, calibrated outputs, API contracts, testing, artifact versioning, monitoring, containerization, and responsible-AI reasoning.

Project 2: Semantic Search and RAG API

- What it does:** Ingests documents asynchronously, creates embeddings, retrieves relevant chunks, answers questions with citations, and streams tokens to clients.
- Tech stack:** FastAPI, Pydantic, a sentence-transformer/embedding API, PostgreSQL with pgvector or a vector database, an LLM provider or local model server, Docker, OpenTelemetry.
- Dataset suggestion:** Public company reports, arXiv papers, course notes, or a curated technical documentation collection with question-answer evaluation pairs.
- Resume value:** Shows async orchestration, background job design, retrieval evaluation, streaming, source attribution, rate limiting, and observability—strong signals for GenAI and AI engineer roles.

Project 3: Image Moderation Inference Gateway

- What it does:** Accepts bounded image uploads, validates file signatures and dimensions, runs moderation/classification, returns labels and confidence, and stores only safe audit metadata.
- Tech stack:** FastAPI `UploadFile`, Pillow/OpenCV, PyTorch or ONNX Runtime, Docker, a queue for slow batches, object storage for explicitly retained samples, pytest and load testing.
- Dataset suggestion:** CIFAR-10 for a harmless prototype, a public content-safety dataset where licensing permits, or a custom labeled dataset with documented consent and governance.
- Resume value:** Demonstrates secure file handling, CV preprocessing consistency, accelerator serving, batch/latency trade-offs, privacy controls, and failure-mode testing.

19. Quick Revision

Item	Revision point
Key idea	FastAPI turns typed Python functions and Pydantic schemas into validated ASGI API endpoints with OpenAPI docs.
Main formula	$L_{total} = L_{network} + L_{queue} + L_{validation} + L_{preprocess} + L_{inference} + L_{postprocess} + L_{serialization}$
Queueing formula	Little's Law: $N = \lambda W$

Item	Revision point
When to use	Python APIs, ML inference, RAG orchestration, typed internal services, streaming and WebSockets
When not enough	Specialized high-throughput GPU serving, durable jobs, model registry, gateway, or orchestration
Important model metrics	Precision, recall, F1, PR-AUC/ROC-AUC, calibration, slice performance
Important service metrics	RPS, p50/p95/p99 latency, 4xx/5xx, queue depth, CPU/GPU/memory, availability
Correct async rule	Await async I/O in <code>async def</code> ; keep blocking work out of the event loop
Model lifecycle	Train offline; load once per process during lifespan; expose readiness and version
Common traps	Model per request, preprocessing skew, too many workers, unlimited payloads, hard-coded secrets, raw tensor serialization
Security	TLS, authentication, authorization, input limits, rate limiting, safe logging, trusted artifacts
Interview one-liner	"FastAPI is the typed API contract and orchestration layer; model execution strategy still determines ML throughput, memory, and latency."

20. Final Cheat Sheet

Definition

FastAPI is a Python ASGI framework that combines Starlette's web capabilities, Pydantic's data validation, and OpenAPI-based documentation.

Input and output

- **Input:** path/query/header values, JSON bodies, forms, files, or streams.
- **Output:** validated JSON, files, streams, status codes, and headers.
- **ML pattern:** validated domain data -> saved preprocessing -> inference -> postprocessing -> versioned response.

Main steps

1. Define Pydantic request and response schemas.
2. Create routes with correct HTTP methods and status behavior.
3. Load trusted model artifacts during lifespan startup.
4. Reuse the training preprocessing pipeline.
5. Add authentication, input limits, and safe errors.
6. Test valid, invalid, unauthorized, unavailable, and boundary cases.
7. Measure model quality and service performance separately.
8. Containerize, deploy with probes, observe, canary, and retain rollback capability.

Key controls

FastAPI does not have model hyperparameters. Important serving controls are:

- Worker/replica count.
- Concurrency limit.
- Timeouts and retry policy.
- Payload, sequence, output, and batch limits.
- Batch size and maximum batch wait.
- CPU/BLAS thread count.
- Model precision and decision threshold.
- Cache policy and rate limits.

Metrics

- **API:** RPS, p50/p95/p99, availability, 4xx/5xx, queue depth, saturation.
- **Resources:** CPU, RAM, GPU utilization, VRAM, disk/network I/O.
- **Model:** task metric, calibration, drift, data quality, slice performance, delayed ground truth.

Pros

- Concise typed request and response contracts.
- Automatic OpenAPI, Swagger UI, and JSON Schema.
- Strong Python/ML library compatibility.
- Async I/O, streaming, WebSockets, middleware, and dependency injection.
- Good testability and a gentle path from prototype to production API.

Cons

- Async does not solve CPU/GPU compute bottlenecks.
- Multiple workers can duplicate large models.
- It is not a durable queue, model registry, feature store, gateway, or specialized GPU scheduler.
- Python and JSON overhead may be unsuitable for some ultra-low-latency internal paths.
- Production quality still requires security, observability, deployment, and data/model governance.

Best use cases

- Online inference for small and medium Python models.
- Business-logic and authentication layer in front of a dedicated model server.
- RAG and LLM orchestration dominated by external I/O.
- Typed internal AI services, prototypes that need a credible production path, and interview projects demonstrating end-to-end ML engineering.

Final interview answer

FastAPI is excellent for exposing Python ML capabilities through validated, documented ASGI endpoints. I load the trusted model once during startup, preserve the training preprocessing pipeline, keep blocking compute off the event loop, version the contract and artifact, secure and limit inputs,

test boundary conditions, and optimize only after measuring end-to-end latency, throughput, memory, and model quality.